

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

SOFTWARE ENGINEERING - LEVEL 3 MINI-PROJECT
SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Traffic Management System (Simulation)

Item	Detail
Project Title	Traffic Management System (Simulation)
Project ID	TBD
Group Number	Team 11
Course	UE24CS341A - Software Engineering
Semester	V Semester
Development Methodology	Agile
Technology	C / C++
Version	1.0
Team Members	Rohan Komperla - PES1UG24CS380 / Rujul Jain - PES1UG24CS388 / Shana Susan Varghese - PES1UG24CS427 / M Ajjaiah - PES1UG25CS823

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional and security requirements for the Traffic Management System (Simulation). The system is a C/C++ based simulation that models vehicle movement through roads and intersections, traffic signal strategies, emergency vehicle priority and traffic incidents.

1.2 Intended Audience

This SRS is intended for the development team, the course faculty/evaluators, and anyone who maintains the project after this phase.

1.3 Product Scope

The system provides a controlled simulation environment to model traffic flow and evaluate fixed-time and adaptive signal strategies. It measures traffic performance indicators such as queue length, waiting time and throughput, and allows different strategies to be compared.

1.4 References

- Team 11 Synopsis / Project Proposal - Traffic Management System (Simulation)
- PES University Software Engineering Level 3 - Project Guidelines 2026

2. Overall Description

2.1 Product Perspective

The system is a standalone simulation application. It does not control real traffic-signal hardware and does not integrate with live traffic sensors or emergency services. All road, vehicle and signal data used by the system is generated or configured within the simulation.

2.2 Product Functions

- Configure road network, lanes and intersections
- Generate vehicles under configurable traffic density
- Simulate vehicle movement and queue formation
- Operate fixed-time and adaptive traffic signal strategies
- Give emergency vehicles priority at signals
- Simulate traffic incidents such as road blockages
- Monitor live traffic conditions during a run
- Generate performance analytics and compare strategies

2.3 User Classes and Characteristics

User Class	Characteristics
Simulation Operator	Configures scenarios, starts/stops runs, introduces incidents. Frequent user during testing.
Traffic Administrator	Configures road network, intersections, signals and strategies. Higher-privilege user.
Traffic Analyst	Reviews traffic metrics and compares strategy performance. Read-focused user.
Development Team	Builds, tests, validates and maintains the simulation.

2.4 Operating Environment

The system runs as a desktop application on a standard Linux/Windows machine, built with C++17 and CMake, with a terminal or lightweight graphical output for simulation state and statistics.

2.5 Design and Implementation Constraints

- Implementation language restricted to C/C++
- CMake used as the build system
- Agile methodology followed throughout development
- Git/GitHub used for version control; Jira used for backlog and sprints

2.6 Assumptions and Dependencies

- All traffic data (vehicle arrival, density, incidents) is simulated, not real-world data
- GoogleTest is available for unit testing
- The simulation runs on a single machine in a single process

3. External Interface Requirements

3.1 User Interfaces

A terminal-based or simple graphical interface displays the road network, live vehicle/queue state, signal states and end-of-run statistics. Configuration is done through a config file or command-line options.

3.2 Software Interfaces

The system does not depend on external databases or third-party services. Internal modules communicate through well-defined C++ class interfaces.

3.3 Communications Interfaces

Not applicable. The simulation runs as a single local process with no network communication.

4. Analysis Models

Use Case Diagram covering all actors and use cases listed in Section 9, and a Class Diagram covering the core domain classes: Road, Intersection, TrafficSignal, Vehicle, IncidentManager and AnalyticsEngine. Diagrams to be added during the Design phase.

5. System Features

Features are grouped by the functional areas identified in the synopsis (F1-F8). Each functional requirement below is uniquely tagged, concise, verifiable and traceable to a use case.

ID	Requirement	Priority	Verification
FR-001	The system shall allow configuration of roads, lanes and intersections before a run starts.	High	Functional test
FR-002	The system shall allow configuration of traffic signals at each intersection.	High	Functional test
FR-003	The system shall generate vehicles according to a configurable traffic-density setting.	High	Functional test
FR-004	The system shall move vehicles along assigned paths and place them in a queue when movement is restricted.	High	System test
FR-005	The system shall support a fixed-time signal strategy with configurable cycle timings.	High	Functional test
FR-006	The system shall support an adaptive signal strategy that adjusts timing based on measured traffic density.	High	Functional test
FR-007	The system shall identify simulated emergency vehicles and grant them signal priority.	High	Integration test
FR-008	The system shall allow a simulated incident to reduce or block capacity on a selected road.	Medium	Functional test
FR-009	The system shall record queue length, waiting time and throughput during a run.	High	Unit + system test
FR-010	The system shall generate summary statistics at the end of a simulation run.	High	Functional test
FR-011	The system shall allow the same scenario to be run under different strategies for comparison.	Medium	System test
FR-012	The system shall reject an invalid or incomplete network configuration with a descriptive error.	Medium	Negative test

6. Non-Functional Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	The simulation shall process at least 1000 vehicle movement steps within 2 seconds on the reference test machine.	Performance test
NFR-002	Reliability	A simulation run shall complete without abnormal termination for any valid configuration.	System test
NFR-003	Maintainability	Core simulation logic shall be separated into independent modules with automated unit tests.	Code review
NFR-004	Testability	Core modules (signal control, vehicle movement, analytics) shall be testable in isolation using mocks/stubs.	Unit test
NFR-005	Usability	An operator shall be able to start a basic simulation run using no more than 3 configuration steps.	Usability walkthrough

7. Security Requirements

ID	Security Requirement	Validation
SEC-001	The system shall validate all configuration file inputs before use.	Negative/security
SEC-002	The system shall reject configuration values outside defined safe ranges (e.g. negative densities).	Negative test
SEC-003	The system shall avoid unsafe memory operations (buffer overflows, unchecked pointers) in core modules.	Static analysis
SEC-004	The system shall log invalid configuration attempts for later	System test

8. Business Rules / Domain Rules

- An emergency vehicle always receives priority over the normal signal cycle at its intersection.
- A simulated incident can only be placed on a road that exists in the configured network.
- Only the Traffic Administrator role may modify the road network or signal configuration during setup.
- Adaptive mode may only use traffic-density data collected within the current run.
- A vehicle cannot proceed through an intersection when its signal is red, except an emergency vehicle under priority rules.

9. Use Cases

ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Configure Road Network	Administrator	Network is not yet defined	Roads, lanes and intersections configured
UC-02	Configure Traffic Signals	Administrator	Road network exists	Signal states and strategy configured
UC-03	Start Simulation	Operator	Configuration complete	Simulation begins execution
UC-04	Generate Traffic	System	Simulation running	Vehicles generated per density setting
UC-05	Simulate Vehicle Movement	System	Vehicles exist	Vehicles move / queue per signal state
UC-06	Manage Adaptive Signals	Traffic Management Module	Adaptive mode enabled	Signal timing adjusted to density
UC-07	Handle Emergency	Operator / System	Emergency vehicle active	Vehicle receives signal priority
UC-08	Simulate Traffic Incident	Operator	Simulation running	Selected road capacity reduced/blocked
UC-09	Monitor Traffic	Analyst / Operator	Simulation running	Live traffic state displayed
UC-10	View Simulation Analytics	Analyst	Run completed	Performance metrics and comparison shown

10. Validation Specification - Initial Planning

At this stage, validation planning identifies how each requirement category will eventually be validated. This is planning only; execution and results are reported later in the project.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests	Configure network, run fixed-time strategy
Security	Input validation / negative tests	Invalid config file, out-of-range density
Performance	Load / timing test	1000+ vehicle-step run under time limit

Requirement Category	Planned Validation	Example
Reliability	Long-run / stress scenarios	Extended run with high density, no crash
Usability	Task-based walkthrough	Operator configures and starts a basic run
Unit	Automated unit tests + coverage	Signal timing calculation, queue operations

11. Project Plan

Sprint	Planned Activities	Acceptance Evidence
Sprint 1	Requirements, GitHub/Jira setup, project structure	SRS baseline, repo, backlog
Sprint 2	Road network and vehicle-flow simulation	Working network + vehicle flow, unit tests
Sprint 3	Traffic signal control (fixed + adaptive)	Working signal strategies, test evidence
Sprint 4	Emergency priority and incident management	Integrated features, validation evidence
Sprint 5	Analytics and module integration	Integrated simulation, performance results
Sprint 6	Validation, CI/CD, static analysis, demo prep	Final validation evidence, release build

11.1 Tool Stack

Activity	Tool	Purpose
Programming	C++17	Implementation of simulation modules
Build System	CMake	Build configuration
Version Control	Git + GitHub	Source control, branches, PRs
Agile Planning	Jira	Backlog, user stories, sprints
Unit Testing	GoogleTest	Automated unit testing
CI/CD	Jenkins	Automated build and test pipeline
Containerization	Docker	Reproducible build environment
Static Analysis	SonarQube or similar	Code quality checks

Activity	Tool	Purpose
Design	Draw.io / PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
Queue	Ordered list of vehicles waiting at an intersection for signal clearance
Fixed-time strategy	Signal strategy where green/red durations are pre-set and do not change
Adaptive strategy	Signal strategy where timing changes based on measured traffic density
Throughput	Number of vehicles that pass through an intersection in a given time
Incident	A simulated event that reduces or blocks road capacity

Appendix B: Field Layouts

Field	Length	Data Type	Description	Mandatory
road_id	10	String	Unique road identifier	Y
intersection_id	10	String	Unique intersection identifier	Y
signal_state	10	Enum	RED / GREEN / YELLOW	Y
vehicle_id	10	String	Unique vehicle identifier	Y
vehicle_type	15	Enum	NORMAL / EMERGENCY	Y
queue_length	5	Integer	Number of vehicles waiting	Y
wait_time	8	Float	Time spent waiting, in seconds	Y
incident_id	10	String	Unique incident identifier	N

Appendix C: Requirement Traceability Matrix

To be completed during the Design phase, once architecture and code references are available.

Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
FR-001	Configure road network					
FR-005	Fixed-time signal strategy					
FR-006	Adaptive signal strategy					
FR-007	Emergency vehicle priority					
FR-009	Traffic metrics collection					

**PES UNIVERSITY, BANGALORE Department of Computer Science and
Engineering SOFTWARE ENGINEERING**

LEVEL 3 MINI-PROJECT DESIGN

TRAFFIC MANAGEMENT SYSTEM (SIMULATION)

TEAM 11

Project Detail	Information
Course	UE24CS341A – Software Engineering
Semester	5
Group Number	Team 11
Development Methodology	Agile
Technology	C / C++
Version	1.0
Academic Year	2026
Project ID	TBD

Team Members

Sl. No.	Student Name	SRN
1	Rohan Komperla	PES1UG24CS380
2	Rujul Jain	PES1UG24CS388
3	Shana Susan Varghese	PES1UG24CS427
4	M Ajjaiah	PES1UG25CS823

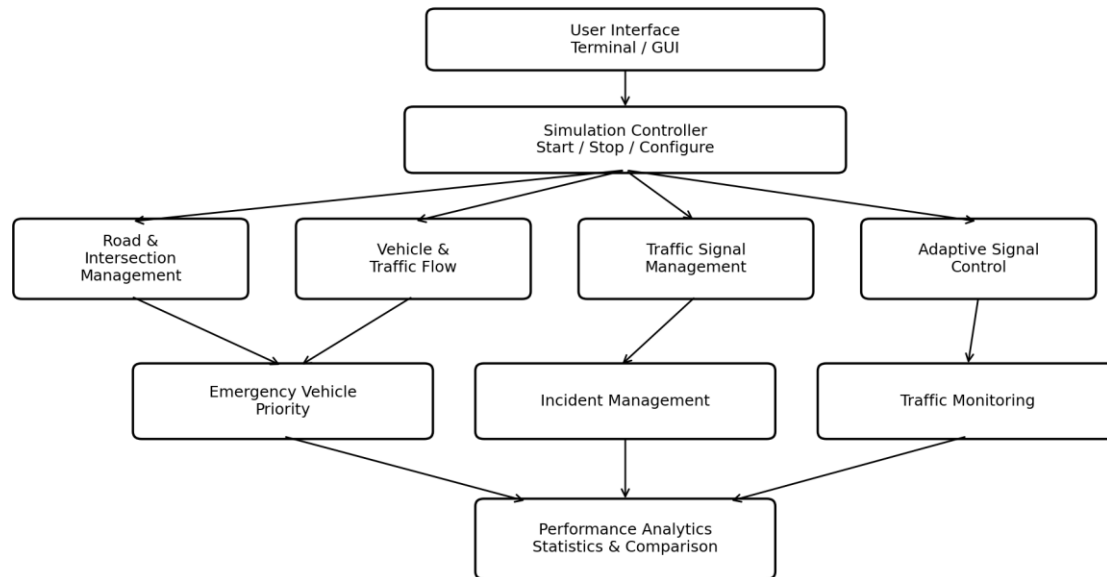
1. Design Overview

This document presents the design of the Traffic Management System (Simulation) based on the approved Team 11 synopsis and SRS. The design translates the specified functional features and use cases into a modular software structure, UML diagrams, interaction flows and requirement traceability.

The SRS identifies Use Case and Class Diagrams as Design-phase artifacts and names Road, Intersection, TrafficSignal, Vehicle, IncidentManager and AnalyticsEngine as the core domain classes. The synopsis identifies eight functional features covering road/intersection management, vehicle flow, traffic signals, adaptive control, emergency priority, incident simulation, monitoring and analytics.

Design note: Where the source documents do not prescribe exact class attributes, method signatures or internal component boundaries, the diagrams below provide a consistent design interpretation for the stated requirements. Such details should be aligned with the team's implementation if the code structure changes.

2. System Architecture



Modular C/C++ simulation design

Figure 1. High-level modular architecture.

The architecture separates user interaction, simulation control, road/intersection management, vehicle flow, signal management, adaptive control, emergency/incident handling, monitoring and analytics. This is consistent with the modular functional ownership and technology plan in the project documents.

3. Use Case Design

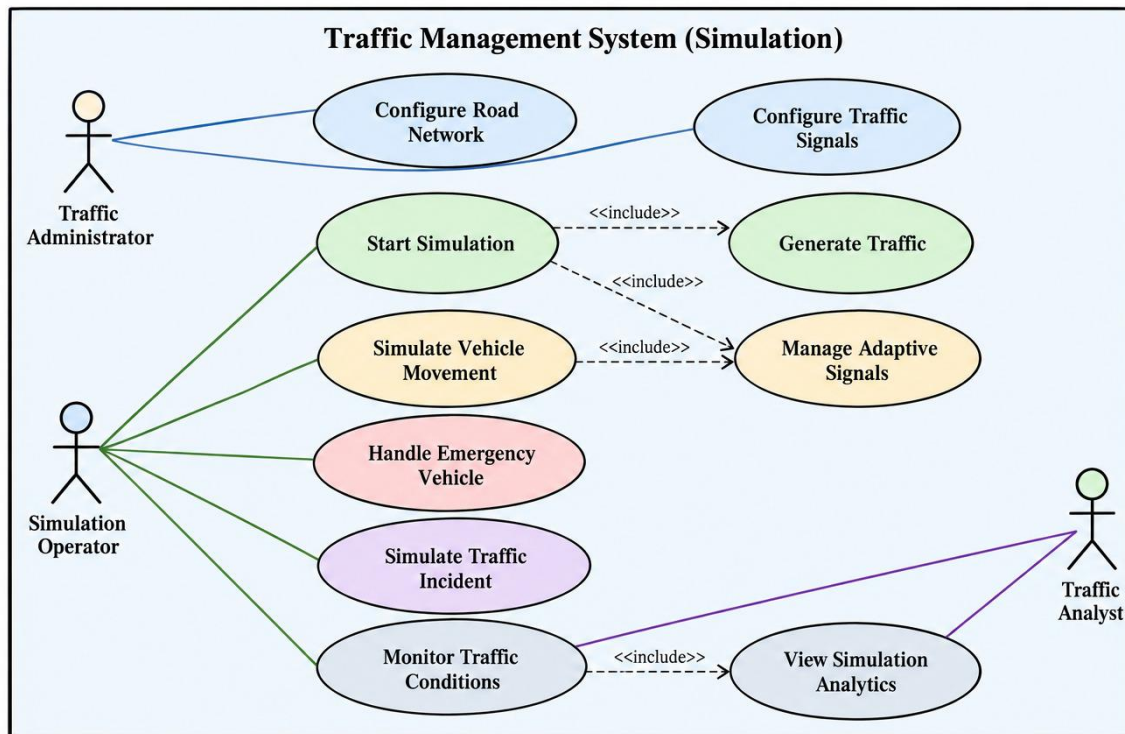


Figure 2. Use Case Diagram.

The diagram covers the ten use cases defined in the SRS: configure road network, configure traffic signals, start simulation, generate traffic, simulate vehicle movement, manage adaptive signals, handle emergency vehicle, simulate traffic incident, monitor traffic and view simulation analytics.

4. Class Design

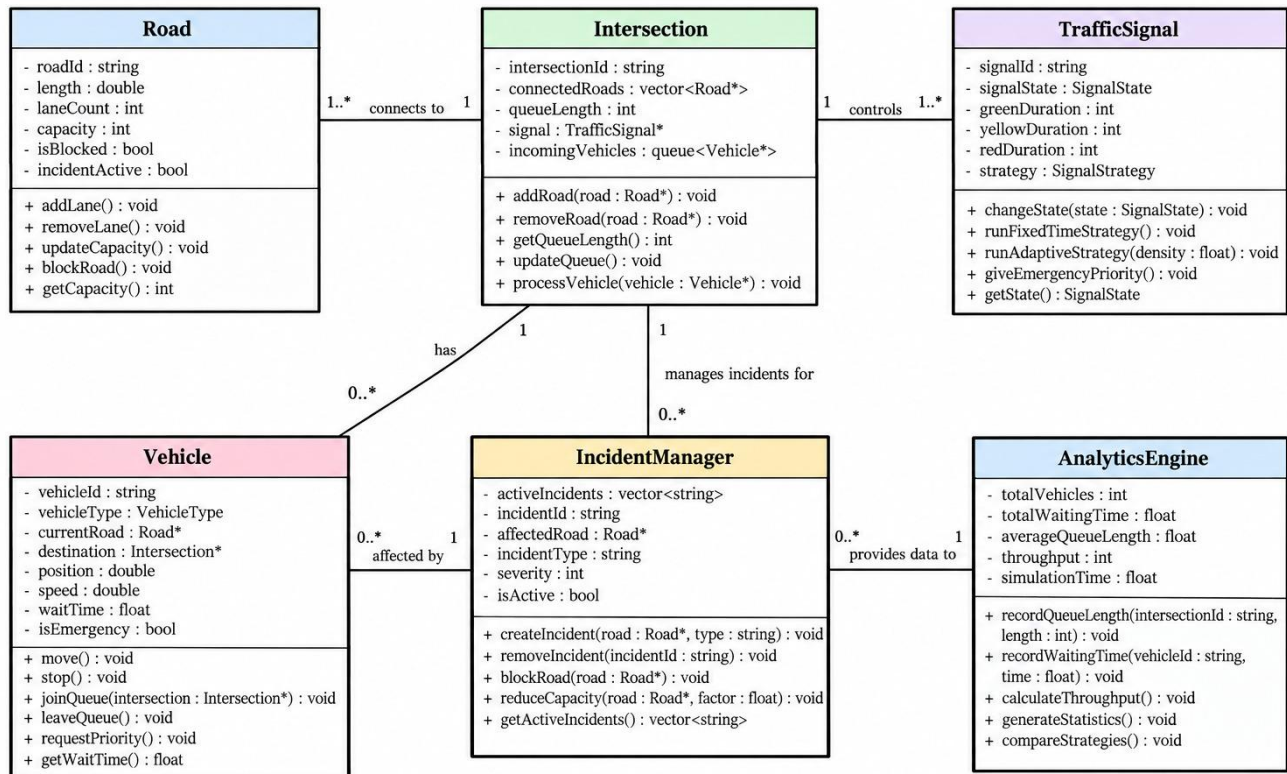


Figure 3. Class Diagram.

The six core classes are taken directly from the SRS. Attributes and operations shown are design-level proposals that support the documented requirements and field layouts; they are not claimed to be existing implementation details.

5. Sequence Design

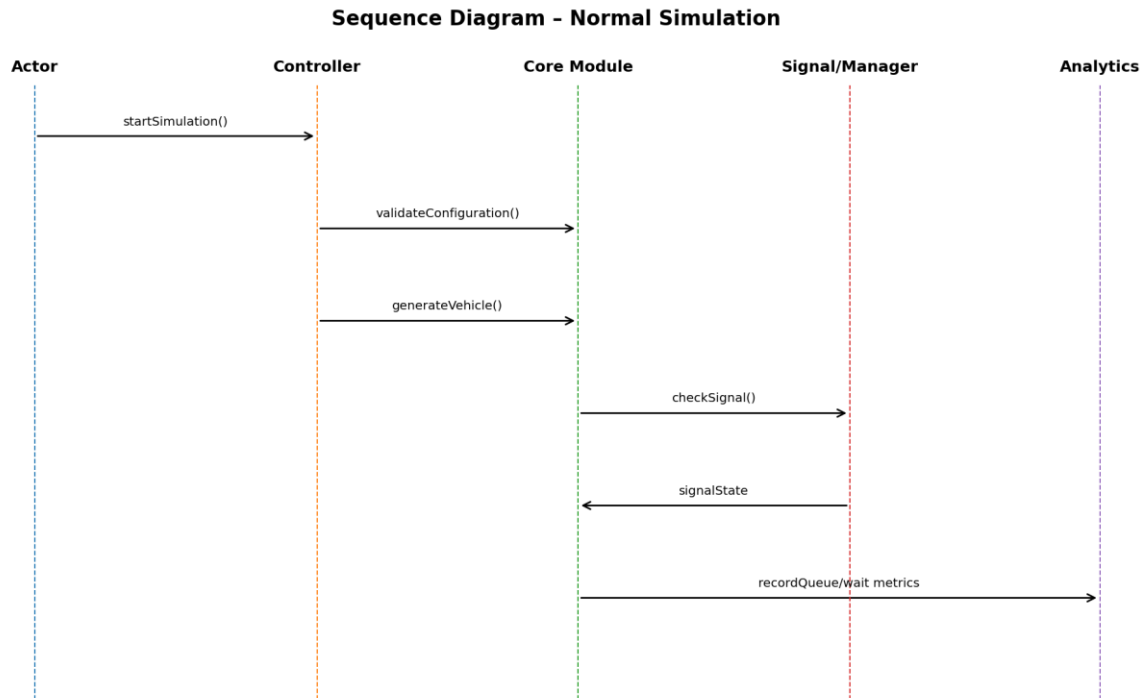


Figure 4. Normal simulation sequence.

The normal flow starts with the operator initiating a run, followed by configuration validation, vehicle generation, signal checking, movement/queue behavior and metric collection.

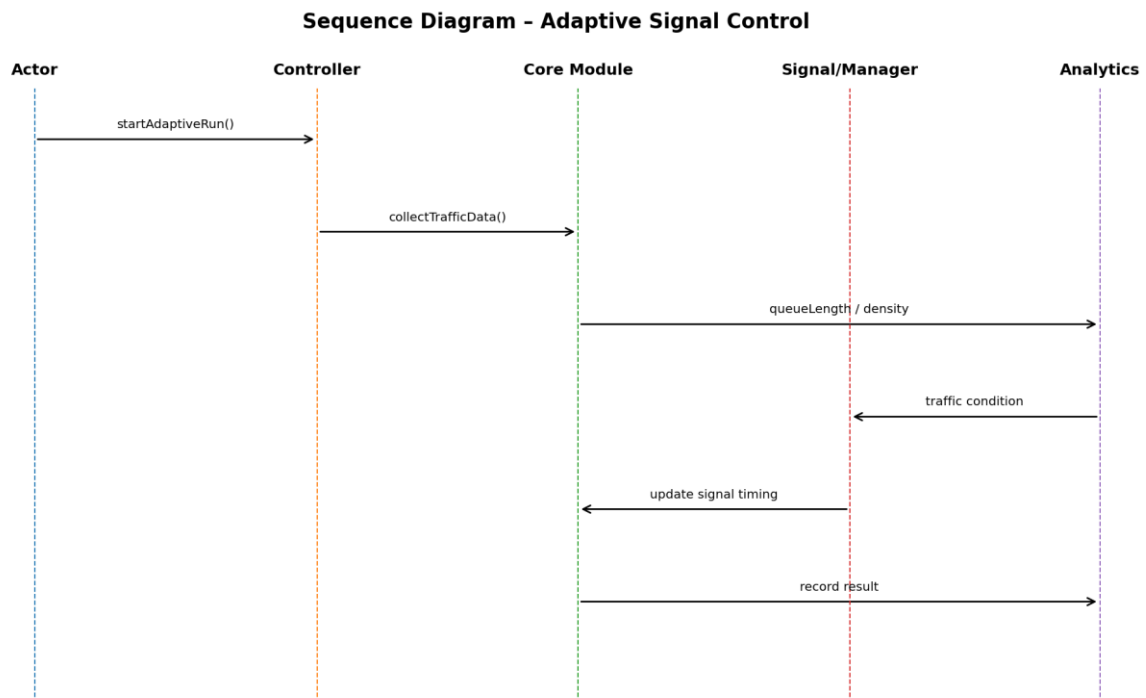


Figure 5. Adaptive signal-control sequence.

Adaptive control uses traffic-density information from the current run to adjust signal operation, as required by FR-006 and the adaptive-strategy domain rule.

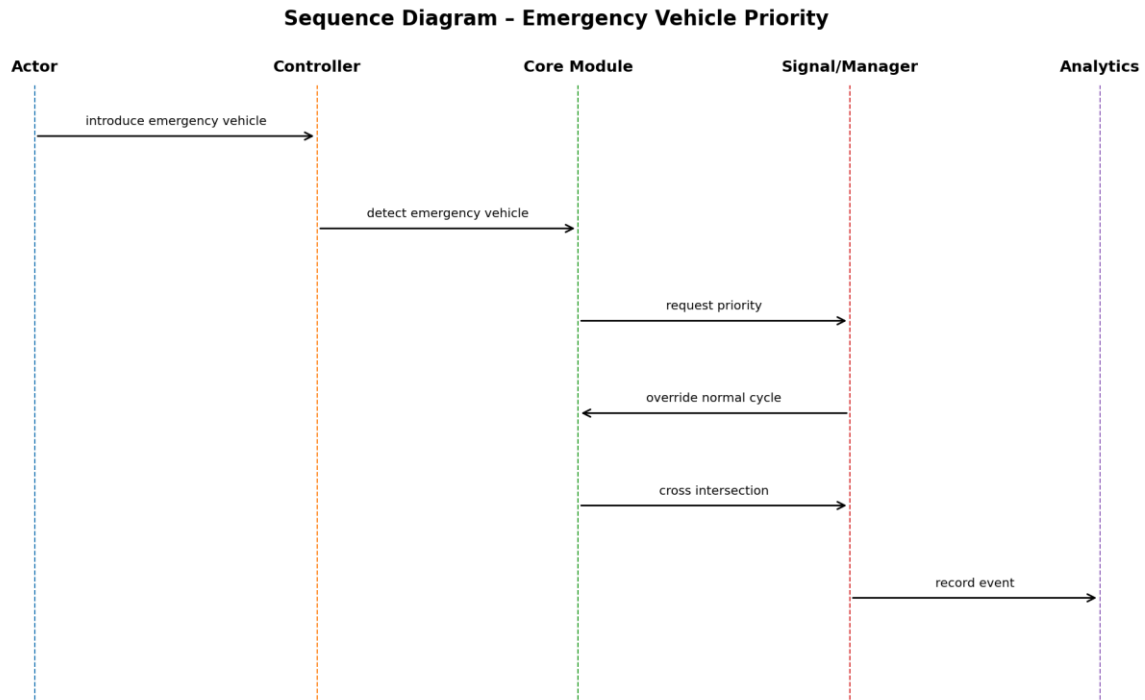


Figure 6. Emergency vehicle priority sequence.

Emergency handling gives the emergency vehicle priority over the normal signal cycle, consistent with the stated business rule.

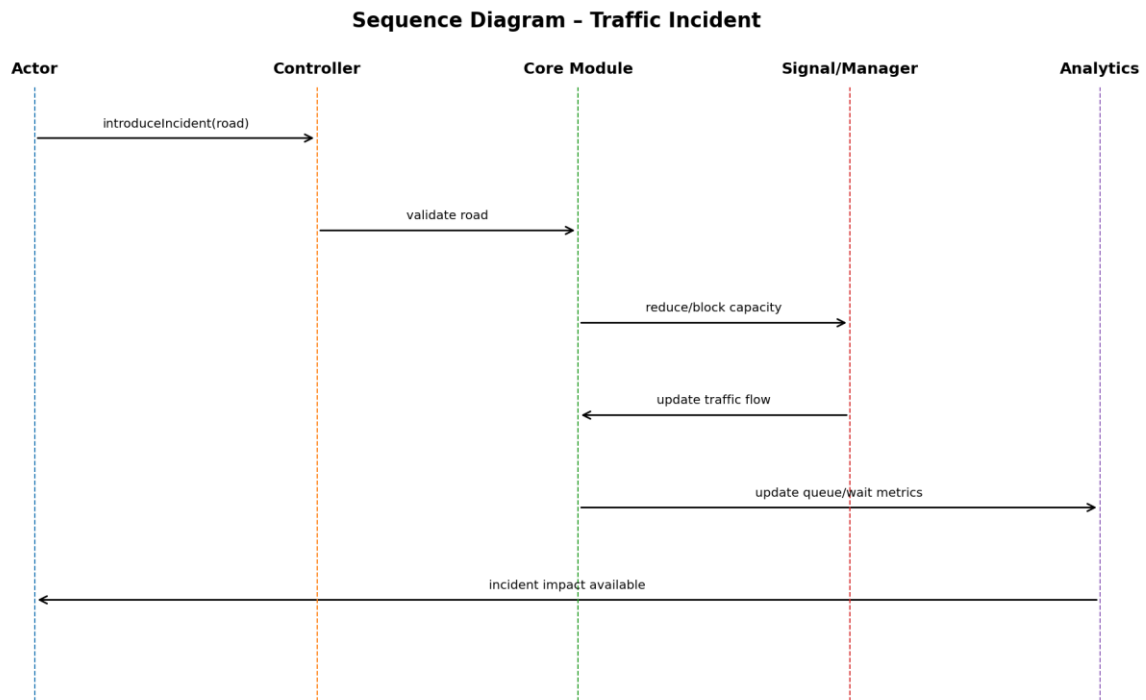


Figure 7. Traffic incident sequence.

Incident handling validates the selected road, reduces or blocks its capacity, updates simulated traffic flow and records the resulting traffic metrics.

6. Activity Design

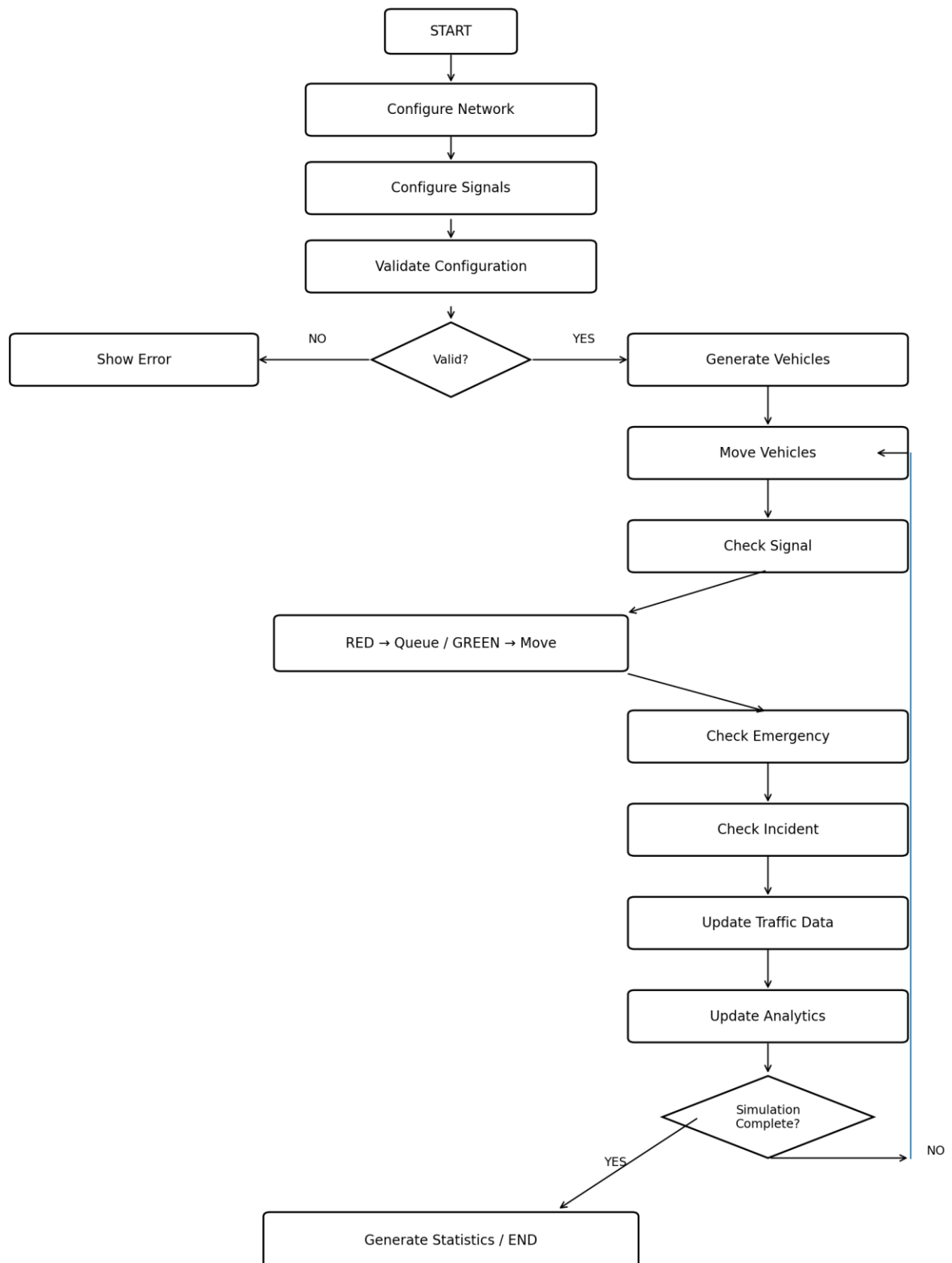


Figure 8. End-to-end simulation activity flow.

The activity flow begins with network and signal configuration, validates the configuration, generates vehicles, processes movement against signal conditions, checks emergency and incident conditions, updates traffic data and analytics, and ends by generating simulation statistics.

7. Requirement Traceability Matrix

Req ID	Requirement	Architecture Ref	Design Ref	Code Ref	Test Case
FR-001	Configure roads, lanes and intersections	Road & Intersection Management	UC-01; Class Diagram	TBD	TC-001
FR-002	Configure traffic signals	Traffic Signal Management	UC-02; TrafficSignal	TBD	TC-002
FR-003	Generate vehicles using traffic density	Vehicle & Traffic Flow	UC-04; Vehicle	TBD	TC-003
FR-004	Move vehicles and form queues	Vehicle & Intersection	UC-05; Vehicle/Intersection	TBD	TC-004
FR-005	Fixed-time signal strategy	Traffic Signal Management	UC-02; TrafficSignal	TBD	TC-005
FR-006	Adaptive signal strategy	Adaptive Signal Control	UC-06; Seq-02	TBD	TC-006
FR-007	Emergency vehicle priority	Emergency Vehicle Priority	UC-07; Seq-03	TBD	TC-007
FR-008	Simulated traffic incidents	Incident Management	UC-08; Seq-04	TBD	TC-008
FR-009	Record queue, waiting time and throughput	Traffic Monitoring / Analytics	UC-09/10; AnalyticsEngine	TBD	TC-009
FR-010	Generate end-of-run statistics	Performance Analytics	UC-10; AnalyticsEngine	TBD	TC-010
FR-011	Compare strategies using same scenario	Performance Analytics	UC-10; AnalyticsEngine	TBD	TC-011
FR-012	Reject invalid/incomplete configuration	Configuration / Validation	UC-01; Activity Diagram	TBD	TC-012

8. Non-Functional Design Mapping

Requirement	Design Response
NFR-001 Performance	Efficient modular simulation loop and queue/vehicle operations; benchmark against stated 1000 vehicle-step / 2 second target.
NFR-002 Reliability	Configuration validation and controlled state transitions; valid runs should complete without abnormal termination.
NFR-003 Maintainability	Separate road, intersection, signal, vehicle, incident and analytics responsibilities.
NFR-004 Testability	Keep core modules independently callable/testable and support mocks/stubs where needed.
NFR-005 Usability	Provide a simple configuration/start workflow through terminal or lightweight GUI.

9. Design Constraints and Rules

- Implementation is restricted to C/C++ and uses CMake.
- All traffic data is simulated; the system does not control real traffic hardware or use live traffic sensors.
- An emergency vehicle receives priority over the normal signal cycle at its intersection.
- A simulated incident can only be placed on an existing configured road.
- Adaptive mode uses traffic-density data collected within the current run.
- A vehicle cannot proceed through an intersection on a red signal except when emergency-priority rules apply.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT SOFTWARE TEST PLAN (STP)

for

Traffic Management System (Simulation)

Item	Detail
Project Title	Traffic Management System (Simulation)
Project ID	TBD
Group Number	Team 11
Course	UE24CS341A – Software Engineering
Semester	V Semester
Development Methodology	Agile
Technology	C / C++ (C++17)
Version	1.0
Team Members	Rohan Komperla – PES1UG24CS380 Rujul Jain – PES1UG24CS388 Shana Susan Varghese – PES1UG24CS427 M Ajjaiah – PES1UG25CS823

1. Introduction

This Software Test Plan defines the testing approach for the Traffic Management System (Simulation). The system is a standalone C/C++ simulation that models roads, intersections, vehicles, traffic signals, emergency priority, incidents and traffic analytics.

The plan is based on the requirements in the SRS and the architecture/design in the SAD. Testing will verify that the implemented system satisfies the functional, non-functional and security requirements.

1.1 Purpose

The purpose of testing is to detect defects early, verify the stated requirements and confirm that the main simulation flows work correctly under normal, boundary and invalid inputs.

1.2 Scope

Testing covers network configuration, traffic signal configuration, vehicle generation and movement, queues, fixed-time and adaptive signal control, emergency vehicle priority, traffic incidents, monitoring, analytics, invalid configuration handling, performance, reliability, usability and basic security checks.

The system does not control real traffic hardware, use live traffic sensors or communicate with external services; these are outside the test scope.

2. Test Objectives

- Verify all functional requirements FR-001 to FR-012.
- Verify the non-functional requirements for performance, reliability, maintainability, testability and usability.
- Verify the security requirements for input validation, safe ranges, memory safety and logging.
- Verify the main use cases and simulation flows defined in the SRS and SAD.
- Confirm that the same scenario can be executed with different signal strategies for comparison.
- Record test results and maintain requirement-to-test traceability.

3. Test Strategy

Test Level	Purpose	Main Coverage
Unit Testing	Test individual modules and core operations in isolation.	Vehicle movement, queue operations, signal timing, analytics.
Integration Testing	Check interaction between connected modules.	Vehicle + intersection, signal control + vehicle flow, emergency handling, incident handling.
System Testing	Verify the complete simulation against requirements.	End-to-end simulation, analytics, configuration validation.
Performance Testing	Check execution time under the stated load.	1000 vehicle movement steps within 2 seconds on the reference machine.
Security Testing	Check invalid and unsafe inputs.	Invalid config files, out-of-range values, memory-safety checks, invalid-attempt logging.
Usability Testing	Check that a basic run can be configured and started simply.	Operator workflow using terminal/lightweight GUI.

4. Test Environment

Item	Planned Environment
Operating System	Linux / Windows
Programming Language	C++17
Build System	CMake
Unit Testing	GoogleTest

Item	Planned Environment
Version Control	Git + GitHub
CI/CD	Jenkins
Static Analysis	SonarQube or similar
Execution	Standalone single-process simulation

The SRS specifies GoogleTest as the unit-testing dependency and a single-machine, single-process execution model. The project tool stack also identifies CMake, Git/GitHub, Jenkins, Docker and static analysis tooling.

5. Test Types and Approach

Test Type	Approach
Functional	Black-box tests using valid configurations and expected simulation behavior.
Negative	Use incomplete, invalid or out-of-range configuration values and verify descriptive rejection.
Boundary	Test minimum/maximum practical traffic density, queue conditions and signal timings.
Regression	Re-run important test cases after changes to core simulation modules.
Performance	Measure execution time for the stated 1000 vehicle-step target.
Reliability / Stress	Run longer simulations with high traffic density and check for abnormal termination.
Security	Validate configuration inputs, safe ranges, memory safety and invalid-attempt logging.
Usability	Perform a simple operator walkthrough from configuration to simulation start.

6. Test Cases

The following test cases correspond directly to the test-case IDs already referenced by the SAD requirement traceability matrix (TC-001 to TC-012).

TC ID	Req ID	Test Scenario	Test Input / Action	Expected Result
TC-001	FR-001	Configure road network	Enter valid roads, lanes and intersections.	Network is accepted and stored before simulation starts.
TC-002	FR-002	Configure traffic signals	Configure signal state and cycle settings for an intersection.	Signal configuration is accepted and applied.
TC-003	FR-003	Generate traffic	Run simulation with a selected traffic-density value.	Vehicles are generated according to the configured density.
TC-004	FR-004	Vehicle movement and queue	Run vehicles toward an intersection with a red signal.	Vehicles stop and form a queue; vehicles move when permitted.
TC-005	FR-005	Fixed-time strategy	Run a valid scenario using fixed signal timings.	Signals follow the configured fixed-time cycle.
TC-006	FR-006	Adaptive strategy	Run a scenario with adaptive mode enabled and varying traffic density.	Signal timing changes using traffic-density data from the current run.
TC-007	FR-007	Emergency priority	Introduce an emergency vehicle at an intersection.	Emergency vehicle receives priority over the normal signal cycle.
TC-008	FR-008	Traffic incident	Introduce an incident on an existing configured road.	Road capacity is reduced/blocked and traffic flow reflects the incident.
TC-009	FR-009	Traffic metrics	Run a simulation and observe collected metrics.	Queue length, waiting time and throughput are recorded.
TC-010	FR-010	End-of-run statistics	Complete a valid simulation run.	Summary statistics are generated at the end of the run.
TC-011	FR-011	Strategy comparison	Run the same scenario with different signal strategies.	Results for the strategies can be compared using recorded metrics.

TC ID	Req ID	Test Scenario	Test Input / Action	Expected Result
TC-012	FR-012	Invalid configuration	Provide incomplete network data or invalid configuration values.	System rejects the configuration and displays a descriptive error.

7. Non-Functional Test Cases

Test ID	Requirement	Test Scenario	Expected Result
NFT-001	NFR-001 Performance	Execute at least 1000 vehicle movement steps and measure execution time.	The stated 1000 vehicle-step workload completes within 2 seconds on the reference test machine.
NFT-002	NFR-002 Reliability	Run a valid configuration for an extended simulation period with high density.	Simulation completes without abnormal termination.
NFT-003	NFR-003 Maintainability	Review module separation and automated unit tests.	Road, intersection, signal, vehicle, incident and analytics responsibilities remain separated and unit tested.
NFT-004	NFR-004 Testability	Run core modules independently using suitable mocks/stubs where required.	Core modules can be tested in isolation.
NFT-005	NFR-005 Usability	Ask an operator to configure and start a basic run.	Basic simulation can be started using no more than 3 configuration steps.

8. Security Testing

Security Test ID	Req ID	Test	Expected Result
SEC-TC-001	SEC-001	Provide malformed or invalid configuration input.	Input is validated before use and invalid data is rejected.
SEC-TC-002	SEC-002	Provide out-of-range values such as negative traffic density.	System rejects the value and reports a descriptive error.
SEC-TC-003	SEC-003	Perform static analysis / code review of core modules.	No unsafe buffer, pointer or similar memory operations are identified in the reviewed core modules.
SEC-TC-004	SEC-004	Trigger an invalid configuration attempt.	The invalid attempt is logged for later review.

9. Boundary and Negative Scenarios

- Empty or incomplete road-network configuration.
- Invalid intersection or road references.
- Negative or otherwise out-of-range traffic-density values.
- Incident placed on a road that does not exist.
- Vehicle movement attempted through a red signal.
- Emergency vehicle introduced while a normal signal cycle is active.
- High traffic density for stress testing.
- Repeated simulation runs using the same scenario for strategy comparison.

10. Entry and Exit Criteria

Entry Criteria	Exit Criteria
Requirements and design documents are available.	All planned high-priority functional tests are executed.
Build completes successfully.	Critical defects affecting core simulation behavior are resolved or documented.

Entry Criteria	Exit Criteria
Test configuration and test data are prepared.	Security, performance and reliability checks are completed.
Unit-test framework is available.	Traceability from requirements to test cases is complete.
Core modules are available for testing.	Test results and unresolved issues are recorded.

11. Defect Reporting and Regression

Any failed test will be recorded with the test case ID, requirement ID, input/configuration used, observed result and expected result. Defects will be tracked through the team's project workflow.

After a defect is fixed, the failed test will be repeated and relevant dependent tests will be rerun to check for regression.

12. Test Deliverables

- Test case list and test data.
- Unit-test results.
- Functional and system-test results.
- Performance and reliability test results.
- Security-test results.
- Requirement traceability matrix.
- Defect records and regression evidence.
- Final test summary for the project.

13. Requirement Traceability Matrix

Req ID	Requirement	Test Case
FR-001	Configure roads, lanes and intersections	TC-001
FR-002	Configure traffic signals	TC-002
FR-003	Generate vehicles using traffic density	TC-003
FR-004	Move vehicles and form queues	TC-004
FR-005	Fixed-time signal strategy	TC-005
FR-006	Adaptive signal strategy	TC-006
FR-007	Emergency vehicle priority	TC-007
FR-008	Simulated traffic incidents	TC-008
FR-009	Record queue, waiting time and throughput	TC-009
FR-010	Generate end-of-run statistics	TC-010
FR-011	Compare strategies using same scenario	TC-011
FR-012	Reject invalid/incomplete configuration	TC-012
NFR-001	Performance	NFT-001
NFR-002	Reliability	NFT-002
NFR-003	Maintainability	NFT-003
NFR-004	Testability	NFT-004
NFR-005	Usability	NFT-005
SEC-001	Validate configuration inputs	SEC-TC-001
SEC-002	Reject unsafe configuration values	SEC-TC-002
SEC-003	Avoid unsafe memory operations	SEC-TC-003

Req ID	Requirement	Test Case
SEC-004	Log invalid configuration attempts	SEC-TC-004

14. References

- Team 11 Software Requirements Specification (SRS) – Traffic Management System (Simulation), Version 1.0.
- Team 11 Software Architecture and Design Specification (SAD) – Traffic Management System (Simulation), Version 1.0.
- PES University Software Engineering Level 3 Mini-Project guidelines.